

```
In [ ]: import marimo as mo
```

# Protein VQ-VAE Diagnosis

タンパク質 VQ-VAE の 3D 再構成 RMSE ( $\sim 2 \text{ \AA}$ ) の原因を特定し、エポック 60 以降の recon loss 上昇の原因を調査する。

## 分析項目:

1. 量子化 vs エンコーダ容量の切り分け（連続再構成 vs VQ再構成）
2. 次元別の 3D RMSE 寄与分析
3. 潜在空間での量子化誤差
4. 誤差 vs ポケットサイズ・記述子の大きさ
5. Codebook の幾何学的構造
6. 入力記述子の分布特性

```
In [ ]: import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn.functional as F

project_root = Path(__file__).resolve().parent.parent
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))

from src.config import CrossDockedConfig, VQVAETrainingConfig
from src.data.descriptors import ComplexDescriptorDataModule
from src.model.vqvae_module import VQVAEModule

plt.rcParams["figure.dpi"] = 120
```

## 1. Load checkpoint and data

```
In [ ]: # List available checkpoints
ckpt_dir = project_root / "checkpoints"
if ckpt_dir.exists():
    ckpts = sorted(ckpt_dir.rglob("*.ckpt"))
    for _p in ckpts:
        print(_p.relative_to(project_root))
else:
    ckpts = sorted(project_root.rglob("*.ckpt"))
    for _p in ckpts[-10:]:
        print(_p.relative_to(project_root))
```

```
checkpoints/vqvae-epoch=02-val/protein_recon=0.7866.ckpt
checkpoints/vqvae-epoch=03-val/protein_recon=0.6128.ckpt
checkpoints/vqvae-epoch=04-val/protein_recon=0.4670.ckpt
```

```
In [ ]: # Edit checkpoint path here
CKPT_PATH = "/home/sakano/git/pocket-conditioned-ligand-gen/pocket-ligand-vq
print(f"Loading: {CKPT_PATH}")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
module = VQVAEModule.load_from_checkpoint(str(CKPT_PATH), map_location=device)
module.eval()
module.to(device)

protein_vqvae = module.protein_vqvae
print(f"Device: {device}")
```

```
Loading: /home/sakano/git/pocket-conditioned-ligand-gen/pocket-ligand-vqvae/
42tfb6kx/checkpoints/vqvae-epoch=19-val/protein_recon=0.1512.ckpt
```

```
/home/sakano/git/pocket-conditioned-ligand-gen/.venv/lib/python3.12/site-pac
kages/torch/nn/modules/transformer.py:392: UserWarning: enable_nested_tensor
is True, but self.use_nested_tensor is False because encoder_layer.norm_firs
t was True
  warnings.warn(
Device: cuda
```

```
In [ ]: config = VQVAETrainingConfig()
data_config = CrossDockedConfig(data_dir=project_root / "data")
dm = ComplexDescriptorDataModule(config, data_config)
dm.setup()

protein_test = dm.protein_test.to(device)
protein_train = dm.protein_train.to(device)
norm_stats = dm.norm_stats
prot_mean = norm_stats["protein_mean"].to(device)
prot_std = norm_stats["protein_std"].to(device)

print(f"Protein test: {protein_test.shape}")
print(f"Protein train: {protein_train.shape}")
```

```
Protein test: torch.Size([7172582, 9])
Protein train: torch.Size([57478530, 9])
```

## 2. 量子化 vs エンコーダ容量の切り分け

エンコーダ → デコーダ の連続再構成（量子化なし）と エンコーダ → **量子化** → デコーダ  
の VQ 再構成を比較し、誤差が量子化由来かエンコーダのキャパシティ不足かを特定する。

- 連続 MSE  $\approx$  VQ MSE → エンコーダ/デコーダ容量がボトルネック
- 連続 MSE  $\ll$  VQ MSE → 量子化がボトルネック

```
In [ ]: @torch.no_grad()
def compare_continuous_vs_vq(
    model: "ProteinStructureVQVAE",
    data: "Tensor",
```

```

    batch_size: int = 4096,
):
    all_continuous_recon = []
    all_vq_recon = []
    all_z = []
    all_z_quantized = []

    for i in range(0, len(data), batch_size):
        batch = data[i : i + batch_size]
        z = model.encoder(batch)
        quantized, _indices, _loss = model.codebook(z)

        # Continuous: encoder -> decoder (skip quantization)
        continuous_recon = model.decoder(z)
        # VQ: encoder -> quantize -> decoder
        vq_recon = model.decoder(quantized)

        all_continuous_recon.append(continuous_recon)
        all_vq_recon.append(vq_recon)
        all_z.append(z)
        all_z_quantized.append(quantized)

    return (
        torch.cat(all_continuous_recon),
        torch.cat(all_vq_recon),
        torch.cat(all_z),
        torch.cat(all_z_quantized),
    )

cont_recon, vq_recon, z_all, z_quantized_all = compare_continuous_vs_vq(
    protein_vqvae, protein_test
)

# Normalized-space MSE
cont_mse_norm = (protein_test - cont_recon).pow(2).mean().item()
vq_mse_norm = (protein_test - vq_recon).pow(2).mean().item()

# Original-scale MSE
orig = (protein_test * prot_std + prot_mean).cpu().numpy()
cont_orig = (cont_recon * prot_std + prot_mean).cpu().numpy()
vq_orig = (vq_recon * prot_std + prot_mean).cpu().numpy()
cont_mse_orig = np.mean((orig - cont_orig) ** 2)
vq_mse_orig = np.mean((orig - vq_orig) ** 2)

print("=== Continuous (no VQ) vs VQ reconstruction ===")
print(f" Continuous MSE (norm):      {cont_mse_norm:.6f}")
print(f" VQ MSE (norm):                {vq_mse_norm:.6f}")
print(f" Ratio (VQ / Continuous):      {vq_mse_norm / cont_mse_norm:.2f}x")
print()
print(f" Continuous RMSE (orig, Å): {np.sqrt(cont_mse_orig):.4f}")
print(f" VQ RMSE (orig, Å):         {np.sqrt(vq_mse_orig):.4f}")
print()
ratio = vq_mse_norm / cont_mse_norm
if ratio > 2.0: # noqa: PLR2004
    print(" → 量子化が主なボトルネック")
elif ratio < 1.5: # noqa: PLR2004

```

```

    print(" → エンコーダ/デコーダ容量がボトルネック")
else:
    print(" → 両方が同程度に寄与")

```

=== Continuous (no VQ) vs VQ reconstruction ===

```

Continuous MSE (norm):    53.456772
VQ MSE (norm):            0.168812
Ratio (VQ / Continuous):  0.00x

```

```

Continuous RMSE (orig, Å): 23.4375
VQ RMSE (orig, Å):        1.3246

```

→ エンコーダ/デコーダ容量がボトルネック

### 3. 次元別の 3D RMSE 寄与

9D 記述子の各次元が 3D 再構成 RMSE にどれだけ寄与するか。

- dim 0-2: CA 位置 (canonical frame) → ポケット全体のスケールに直接影響
- dim 3-5: N-CA オフセット → 局所的なバックボーン方向
- dim 6-8: C-CA オフセット → 局所的なバックボーン方向

CA 位置の誤差が支配的なら、ポケットの広がり（空間スケール）が問題。オフセットの誤差が大きいなら、局所構造の再構成が困難。

```

In [ ]: dim_labels = [
    "CA_x",
    "CA_y",
    "CA_z",
    "N-CA_x",
    "N-CA_y",
    "N-CA_z",
    "C-CA_x",
    "C-CA_y",
    "C-CA_z",
]
group_labels = ["CA position", "N-CA offset", "C-CA offset"]

per_dim_mse = np.mean((orig - vq_orig) ** 2, axis=0)
per_dim_rmse = np.sqrt(per_dim_mse)

# Group by CA / N-CA / C-CA
group_mse = [
    per_dim_mse[0:3].sum(),
    per_dim_mse[3:6].sum(),
    per_dim_mse[6:9].sum(),
]
group_rmse = [np.sqrt(m) for m in group_mse]

_fig, _axes = plt.subplots(1, 3, figsize=(16, 4))

# Per-dimension MSE
colors = ["#e74c3c"] * 3 + ["#3498db"] * 3 + ["#2ecc71"] * 3

```

```

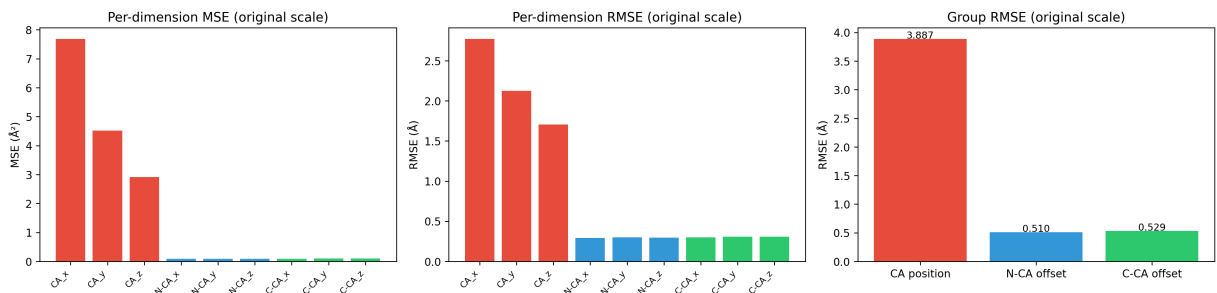
_axes[0].bar(range(9), per_dim_mse, color=colors)
_axes[0].set_xticks(range(9), dim_labels, rotation=45, ha="right", fontsize=
_axes[0].set_ylabel("MSE (Å²)")
_axes[0].set_title("Per-dimension MSE (original scale)")

# Per-dimension RMSE
_axes[1].bar(range(9), per_dim_rmse, color=colors)
_axes[1].set_xticks(range(9), dim_labels, rotation=45, ha="right", fontsize=
_axes[1].set_ylabel("RMSE (Å)")
_axes[1].set_title("Per-dimension RMSE (original scale)")

# Grouped RMSE
bar_colors = ["#e74c3c", "#3498db", "#2ecc71"]
_axes[2].bar(range(3), group_rmse, color=bar_colors)
_axes[2].set_xticks(range(3), group_labels)
_axes[2].set_ylabel("RMSE (Å)")
_axes[2].set_title("Group RMSE (original scale)")
for j, v in enumerate(group_rmse):
    _axes[2].text(j, v + 0.01, f"{v:.3f}", ha="center", fontsize=9)

_fig.tight_layout()
_fig

```



## 4. 潜在空間での量子化誤差

エンコーダ出力  $z$  (L2正規化後) と最近傍コードブックベクトルの距離分布。距離が大きいほど量子化で多くの情報が失われている。

```

In [ ]: # z_all is raw encoder output, z_quantized_all includes L2 norm + scale
# Compute distance on the unit sphere (same space as codebook)
z_norm = F.normalize(z_all, p=2, dim=-1)
# z_quantized_all has scale applied; remove it for fair comparison
z_q_norm = F.normalize(z_quantized_all, p=2, dim=-1)

# Cosine similarity
cosine_sim = (z_norm * z_q_norm).sum(dim=-1).cpu().numpy()
# L2 distance on unit sphere
l2_dist = (z_norm - z_q_norm).pow(2).sum(dim=-1).sqrt().cpu().numpy()

_fig, _axes = plt.subplots(1, 2, figsize=(12, 4))

_axes[0].hist(cosine_sim, bins=100, edgecolor="none", alpha=0.8)
_axes[0].set_xlabel("Cosine similarity (z, quantized_z)")
_axes[0].set_ylabel("Count")

```

```

_axes[0].set_title("Encoder-Codebook cosine similarity")
_axes[0].axvline(
    np.median(cosine_sim),
    color="r",
    linestyle="--",
    label=f"median={np.median(cosine_sim):.4f}",
)
_axes[0].legend(fontsize=8)

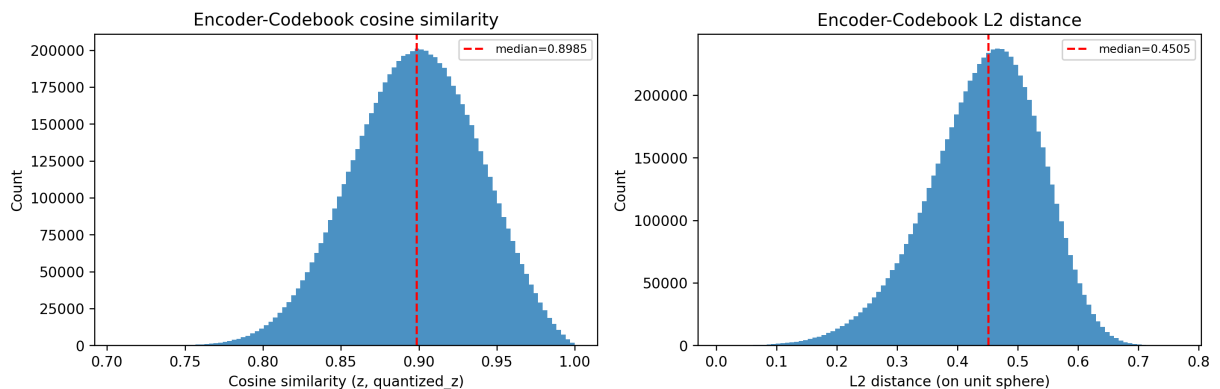
_axes[1].hist(l2_dist, bins=100, edgecolor="none", alpha=0.8)
_axes[1].set_xlabel("L2 distance (on unit sphere)")
_axes[1].set_ylabel("Count")
_axes[1].set_title("Encoder-Codebook L2 distance")
_axes[1].axvline(
    np.median(l2_dist),
    color="r",
    linestyle="--",
    label=f"median={np.median(l2_dist):.4f}",
)
_axes[1].legend(fontsize=8)

_fig.tight_layout()

print(
    f"Cosine sim - mean: {cosine_sim.mean():.4f}, "
    f"P5: {np.percentile(cosine_sim, 5):.4f}, "
    f"P95: {np.percentile(cosine_sim, 95):.4f}"
)
print(
    f"L2 dist - mean: {l2_dist.mean():.4f}, "
    f"P5: {np.percentile(l2_dist, 5):.4f}, "
    f"P95: {np.percentile(l2_dist, 95):.4f}"
)
_fig

```

Cosine sim - mean: 0.8975, P5: 0.8300, P95: 0.9616  
 L2 dist - mean: 0.4431, P5: 0.2773, P95: 0.5830



## 5. 誤差 vs 記述子の大きさ

CA座標の大きさ (centroidからの距離) vs 再構成誤差の関係を調べる。ポケットの端に位置する残基ほど誤差が大きい場合、正規化やcodebook容量に問題がある可能性がある。

```

In [ ]: # Per-residue error
per_residue_mse = np.mean((orig - vq_orig) ** 2, axis=1) # (N,)
per_residue_rmse = np.sqrt(per_residue_mse)

# CA distance from centroid (in original scale, canonical frame)
ca_dist = np.sqrt(np.sum(orig[:, :3] ** 2, axis=1)) # CA_x^2 + CA_y^2 + CA_z^2

# N-CA bond length (original scale)
n_ca_len = np.sqrt(np.sum(orig[:, 3:6] ** 2, axis=1))

_fig, _axes = plt.subplots(1, 3, figsize=(16, 4))

# Error vs CA distance
_axes[0].scatter(ca_dist, per_residue_rmse, s=1, alpha=0.1)
_axes[0].set_xlabel("CA distance from centroid (Å)")
_axes[0].set_ylabel("Per-residue RMSE (Å)")
_axes[0].set_title("RMSE vs CA distance from centroid")
# Add binned trend line
bins = np.linspace(ca_dist.min(), np.percentile(ca_dist, 99), 20)
bin_idx = np.digitize(ca_dist, bins)
min_bin_count = 10
bin_means = [
    np.mean(per_residue_rmse[bin_idx == i])
    for i in range(1, len(bins))
    if np.sum(bin_idx == i) > min_bin_count
]
bin_centers = [
    (bins[i] + bins[i + 1]) / 2
    for i in range(len(bins) - 1)
    if np.sum(bin_idx == i + 1) > min_bin_count
]
_axes[0].plot(bin_centers, bin_means, "r-", linewidth=2, label="binned mean")
_axes[0].legend(fontsize=8)

# Error vs N-CA bond length
_axes[1].scatter(n_ca_len, per_residue_rmse, s=1, alpha=0.1)
_axes[1].set_xlabel("N-CA offset magnitude (Å)")
_axes[1].set_ylabel("Per-residue RMSE (Å)")
_axes[1].set_title("RMSE vs N-CA offset magnitude")

# Per-residue RMSE distribution
_axes[2].hist(per_residue_rmse, bins=100, edgecolor="none", alpha=0.8)
_axes[2].axvline(
    np.median(per_residue_rmse),
    color="r",
    linestyle="--",
    label=f"median={np.median(per_residue_rmse):.3f}",
)
_axes[2].axvline(
    np.percentile(per_residue_rmse, 95),
    color="orange",
    linestyle="--",
    label=f"P95={np.percentile(per_residue_rmse, 95):.3f}",
)
_axes[2].set_xlabel("Per-residue RMSE (Å)")

```

```

_axes[2].set_ylabel("Count")
_axes[2].set_title("Per-residue RMSE distribution")
_axes[2].legend(fontsize=8)

_fig.tight_layout()

print(
    f"Correlation (CA dist, RMSE): {np.corrcoef(ca_dist, per_residue_rmse)[0][1]}
)
_fig

```

/tmp/marimo\_3808627/\_\_marimo\_\_cell\_iLit.py:60: UserWarning: Creating legend with loc="best" can be slow with large amounts of data.

```
_fig.tight_layout()
```

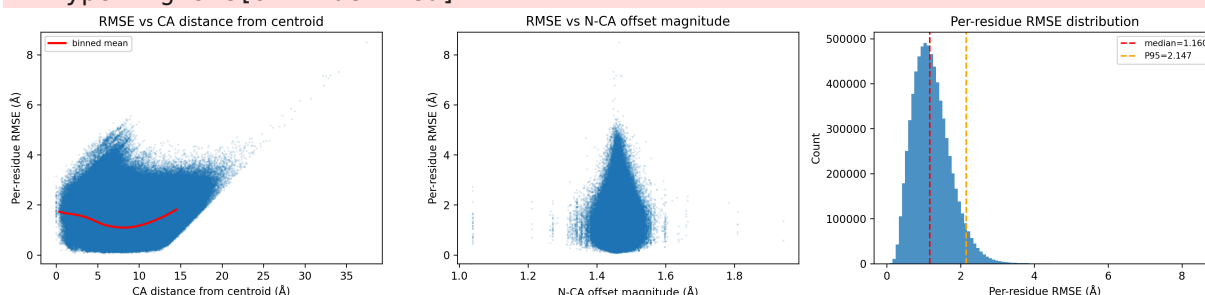
Correlation (CA dist, RMSE): 0.1706

/home/sakano/git/pocket-conditioned-ligand-gen/.venv/lib/python3.12/site-packages/marimo/\_output/mpl.py:90: UserWarning: Creating legend with loc="best" can be slow with large amounts of data.

```

fig.figure.savefig(buf, format="png", bbox_inches="tight", dpi=retina_dpi)
# type: ignore[attr-defined]

```



## 6. 複合体レベルの 3D RMSE 分解

テストセットの複合体をサンプリングし、各複合体ごとに CA位置誤差とオフセット誤差の内訳を3D座標空間で確認する。ポケットサイズ（残基数）との関係も調べる。

```

In [ ]: from src.config import PocketExtractionConfig
        from src.data.descriptors import _parse_types_file
        from src.tokenizers.ligand import parse_sdf
        from src.tokenizers.protein import PocketDescriptor, extract_pocket

pocket_config = PocketExtractionConfig()
desc_calc = PocketDescriptor()
types_file = project_root / "data" / "types" / "cdonly_it2_tt_v1.3_0_test0.t
pairs = _parse_types_file(types_file)
crossdocked_dir = project_root / "data" / "CrossDocked2020"
p_mean = norm_stats["protein_mean"].to(device)
p_std = norm_stats["protein_std"].to(device)

N_SAMPLES = 300
rng = np.random.default_rng(42)
sample_indices = rng.choice(
    len(pairs), size=min(N_SAMPLES * 5, len(pairs)), replace=False
)

```



```

complex_results = []
n_done = 0
for _idx in sample_indices:
    if n_done >= N_SAMPLES:
        break
    rec_rel, lig_rel = pairs[_idx]
    rec_path = crossdocked_dir / rec_rel
    lig_path = crossdocked_dir / lig_rel
    if not rec_path.exists() or not lig_path.exists():
        continue
    try:
        molecules = parse_sdf(lig_path)
        if not molecules:
            continue
        mol = molecules[0]
        lig_coords = np.array(
            [(a[1], a[2], a[3]) for a in mol["atoms"] if a[0] != "H"],
            dtype=np.float32,
        )
        pocket_result = extract_pocket(rec_path, lig_coords, pocket_config)
        if pocket_result is None:
            continue
        backbone_orig, _seq = pocket_result
        prot_desc, prot_meta = desc_calc.compute(backbone_orig)
        n_res = len(prot_desc)

        # VQ reconstruction
        prot_t = torch.from_numpy(prot_desc).to(device)
        prot_norm = (prot_t - p_mean) / p_std
        with torch.no_grad():
            z = protein_vqvae.encoder(prot_norm)
            quantized, indices, _ = protein_vqvae.codebook(z)
            vq_recon_norm = protein_vqvae.decoder(quantized)
            # Continuous reconstruction
            cont_recon_norm = protein_vqvae.decoder(z)

        vq_desc = (vq_recon_norm * p_std + p_mean).cpu().numpy()
        cont_desc = (cont_recon_norm * p_std + p_mean).cpu().numpy()

        # 3D coordinate reconstruction
        backbone_vq = PocketDescriptor.descriptor_to_backbone_coords(
            vq_desc, prot_meta
        )
        backbone_cont = PocketDescriptor.descriptor_to_backbone_coords(
            cont_desc, prot_meta
        )

        # Per-atom RMSE (all 3 backbone atoms per residue)
        vq_rmse = np.sqrt(np.mean((backbone_orig - backbone_vq) ** 2))
        cont_rmse = np.sqrt(np.mean((backbone_orig - backbone_cont) ** 2))

        # CA-only RMSE
        ca_vq_rmse = np.sqrt(
            np.mean((backbone_orig[:, 1] - backbone_vq[:, 1]) ** 2)
        )
        ca_cont_rmse = np.sqrt(

```

```

        np.mean((backbone_orig[:, 1] - backbone_cont[:, 1]) ** 2)
    )

    # Spatial extent of pocket
    ca_coords = backbone_orig[:, 1] # (N, 3)
    pocket_extent = np.max(np.ptp(ca_coords, axis=0))

    complex_results.append(
        {
            "n_res": n_res,
            "vq_rmse": vq_rmse,
            "cont_rmse": cont_rmse,
            "ca_vq_rmse": ca_vq_rmse,
            "ca_cont_rmse": ca_cont_rmse,
            "pocket_extent": pocket_extent,
            "indices": indices.cpu().numpy(),
        }
    )
    n_done += 1
except Exception: # noqa: BLE001, S112
    continue

print(f"Evaluated {len(complex_results)} complexes")

n_res_arr = np.array([r["n_res"] for r in complex_results])
vq_rmse_arr = np.array([r["vq_rmse"] for r in complex_results])
cont_rmse_arr = np.array([r["cont_rmse"] for r in complex_results])
ca_vq_rmse_arr = np.array([r["ca_vq_rmse"] for r in complex_results])
extent_arr = np.array([r["pocket_extent"] for r in complex_results])

_fig, _axes = plt.subplots(2, 2, figsize=(12, 10))

# VQ vs Continuous RMSE
_axes[0, 0].scatter(cont_rmse_arr, vq_rmse_arr, s=10, alpha=0.5)
lim = max(cont_rmse_arr.max(), vq_rmse_arr.max()) * 1.05
_axes[0, 0].plot([0, lim], [0, lim], "r--", linewidth=0.8)
_axes[0, 0].set_xlabel("Continuous RMSE (Å)")
_axes[0, 0].set_ylabel("VQ RMSE (Å)")
_axes[0, 0].set_title("VQ vs Continuous 3D RMSE per complex")
_axes[0, 0].set_aspect("equal", adjustable="datalim")

# RMSE vs pocket size
_axes[0, 1].scatter(n_res_arr, vq_rmse_arr, s=10, alpha=0.5, label="VQ")
_axes[0, 1].scatter(n_res_arr, cont_rmse_arr, s=10, alpha=0.5, label="Conti")
_axes[0, 1].set_xlabel("Pocket size (# residues)")
_axes[0, 1].set_ylabel("3D RMSE (Å)")
_axes[0, 1].set_title("RMSE vs pocket size")
_axes[0, 1].legend(fontsize=8)

# RMSE vs pocket spatial extent
_axes[1, 0].scatter(extent_arr, vq_rmse_arr, s=10, alpha=0.5, label="VQ")
_axes[1, 0].scatter(extent_arr, cont_rmse_arr, s=10, alpha=0.5, label="Conti")
_axes[1, 0].set_xlabel("Pocket spatial extent (Å)")
_axes[1, 0].set_ylabel("3D RMSE (Å)")
_axes[1, 0].set_title("RMSE vs pocket spatial extent")
_axes[1, 0].legend(fontsize=8)

```

```

# Histogram: VQ RMSE breakdown
_axes[1, 1].hist(vq_rmse_arr, bins=30, alpha=0.6, label="All backbone")
_axes[1, 1].hist(ca_vq_rmse_arr, bins=30, alpha=0.6, label="CA only")
_axes[1, 1].set_xlabel("RMSE (Å)")
_axes[1, 1].set_ylabel("Count")
_axes[1, 1].set_title("VQ 3D RMSE: all backbone vs CA only")
_axes[1, 1].legend(fontsize=8)

_fig.tight_layout()

print()
print(
    f"VQ RMSE          - mean: {vq_rmse_arr.mean():.3f}, median: {np.median(vq_rmse_arr):.3f}"
)
print(
    f"Continuous RMSE - mean: {cont_rmse_arr.mean():.3f}, median: {np.median(cont_rmse_arr):.3f}"
)
print(
    f"CA-only VQ RMSE - mean: {ca_vq_rmse_arr.mean():.3f}, median: {np.median(ca_vq_rmse_arr):.3f}"
)
print(
    f"Ratio VQ/Cont    - mean: {(vq_rmse_arr / np.clip(cont_rmse_arr, 1e-6, None)).mean():.3f}"
)
print(
    f"Corr(pocket_size, VQ RMSE): {np.corrcoef(n_res_arr, vq_rmse_arr)[0, 1]:.3f}"
)
print(
    f"Corr(extent, VQ RMSE):      {np.corrcoef(extent_arr, vq_rmse_arr)[0, 1]:.3f}"
)
_fig

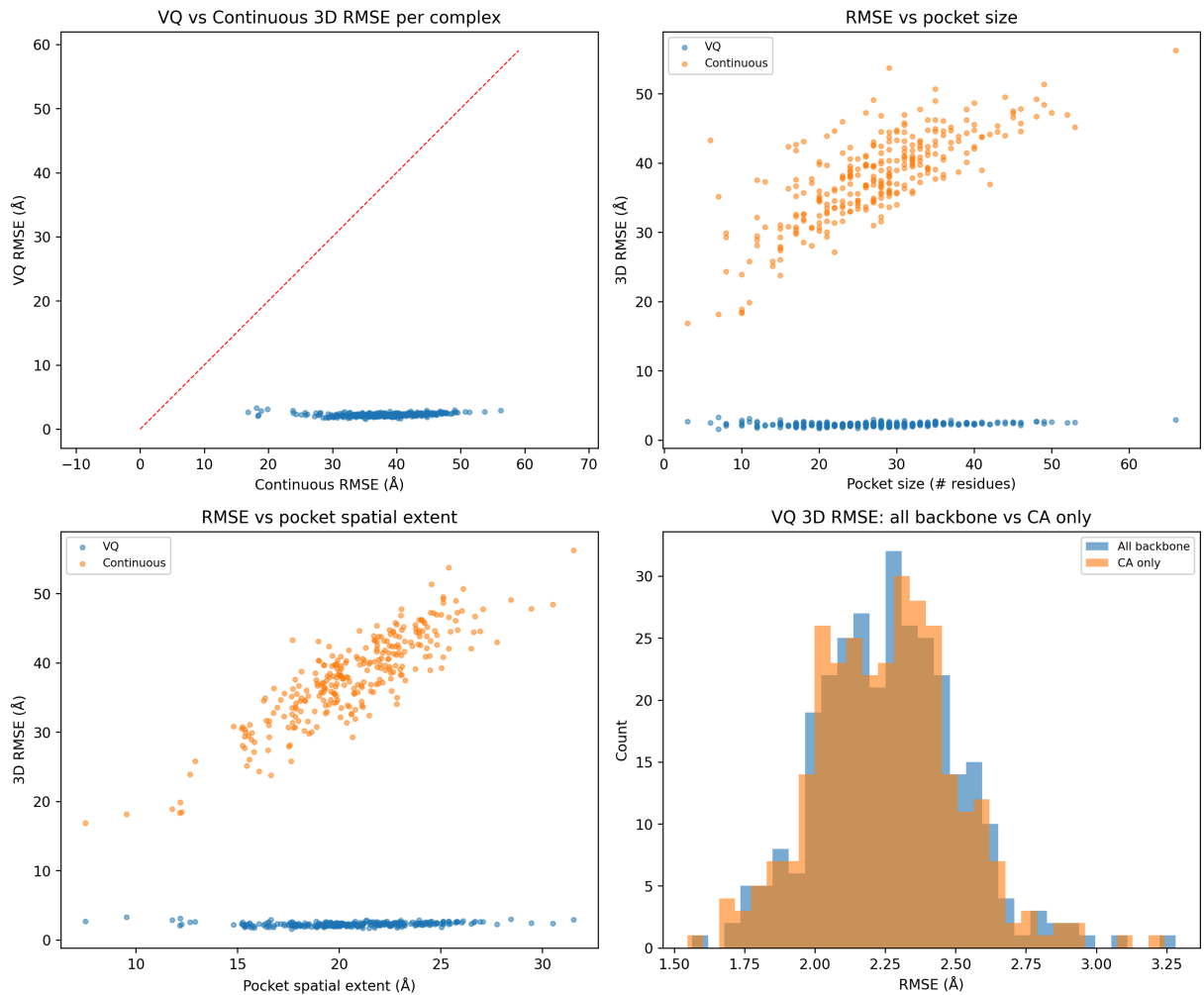
```

Evaluated 300 complexes

```

VQ RMSE          - mean: 2.268, median: 2.275
Continuous RMSE - mean: 38.008, median: 38.258
CA-only VQ RMSE - mean: 2.253, median: 2.264
Ratio VQ/Cont    - mean: 0.06
Corr(pocket_size, VQ RMSE): 0.2342
Corr(extent, VQ RMSE):      0.2222

```



## 7. Codebook の幾何学的構造

コードブックベクトル間の距離分布と、エンコーダ出力から最近傍・2番目近傍コードまでの距離差を調べる。距離差が小さいと、わずかな摂動で異なるコードに割り当てられるため量子化が不安定になる。

```
In [ ]: # Codebook vectors (L2-normalized)
cb = F.normalize(protein_vqvae.codebook.embedding, p=2, dim=-1).cpu().detach()
n_codes = cb.shape[0]

# Pairwise distances between codebook vectors
cb_dists = torch.cdist(cb, cb)
# Set diagonal to inf for min distance
cb_dists_no_diag = cb_dists + torch.eye(n_codes) * 1e6
min_inter_code_dist = cb_dists_no_diag.min(dim=1).values.numpy()

# Encoder output: top-1 and top-2 nearest code distances
z_norm_cpu = F.normalize(z_all, p=2, dim=-1).cpu()
# Subsample for efficiency
rng_cb = np.random.default_rng(42)
n_sub = min(50000, len(z_norm_cpu))
sub_idx = rng_cb.choice(len(z_norm_cpu), n_sub, replace=False)
z_sub = z_norm_cpu[sub_idx]
```

```

dists_to_cb = torch.cdist(z_sub, cb) # (n_sub, n_codes)
top2_dists, _top2_idx = dists_to_cb.topk(2, dim=1, largest=False)
margin = (top2_dists[:, 1] - top2_dists[:, 0]).numpy()

_fig, _axes = plt.subplots(2, 2, figsize=(12, 10))

# Min inter-code distance distribution
_axes[0, 0].hist(min_inter_code_dist, bins=50, edgecolor="none", alpha=0.8)
_axes[0, 0].set_xlabel("Min distance to nearest code")
_axes[0, 0].set_ylabel("Count")
_axes[0, 0].set_title("Codebook: min inter-code distance")
_axes[0, 0].axvline(
    np.median(min_inter_code_dist),
    color="r",
    linestyle="--",
    label=f"median={np.median(min_inter_code_dist):.4f}",
)
_axes[0, 0].legend(fontsize=8)

# Nearest-code distance from encoder outputs
_axes[0, 1].hist(top2_dists[:, 0].numpy(), bins=100, edgecolor="none", alpha=0.8)
_axes[0, 1].set_xlabel("Distance to nearest code")
_axes[0, 1].set_ylabel("Count")
_axes[0, 1].set_title("Encoder output: distance to nearest code")
_axes[0, 1].axvline(
    np.median(top2_dists[:, 0].numpy()),
    color="r",
    linestyle="--",
    label=f"median={np.median(top2_dists[:, 0].numpy()):.4f}",
)
_axes[0, 1].legend(fontsize=8)

# Margin (2nd nearest - nearest) distribution
_axes[1, 0].hist(margin, bins=100, edgecolor="none", alpha=0.8)
_axes[1, 0].set_xlabel("Margin (d2 - d1)")
_axes[1, 0].set_ylabel("Count")
_axes[1, 0].set_title("Quantization margin (larger = more stable)")
_axes[1, 0].axvline(
    np.median(margin),
    color="r",
    linestyle="--",
    label=f"median={np.median(margin):.4f}",
)
_axes[1, 0].legend(fontsize=8)

# Codebook usage count
cb_usage = protein_vqvae.codebook.usage_count.cpu().numpy()
sorted_usage = np.sort(cb_usage)[::-1]
_axes[1, 1].bar(range(len(sorted_usage)), sorted_usage, width=1.0)
_axes[1, 1].set_xlabel("Code rank")
_axes[1, 1].set_ylabel("Usage count (training)")
_axes[1, 1].set_title("Codebook usage (sorted)")
_axes[1, 1].set_yscale("log")

_fig.tight_layout()

```

```

print(
    f"Inter-code dist - min: {min_inter_code_dist.min():.4f}, "
    f"median: {np.median(min_inter_code_dist):.4f}, "
    f"max: {min_inter_code_dist.max():.4f}"
)
print(
    f"Margin (d2-d1) - mean: {margin.mean():.4f}, "
    f"P5: {np.percentile(margin, 5):.4f}, "
    f"P50: {np.percentile(margin, 50):.4f}"
)
print(f"Dead codes (usage=0): {np.sum(cb_usage == 0)}")
print(
    f"Learnable scale (exp(log_scale)): {protein_vqvae.codebook.log_scale.exp}"
)
_fig

```

/tmp/marimo\_3808627/\_\_marimo\_\_cell\_TqIu.py:71: UserWarning: Data has no positive values, and therefore cannot be log-scaled.

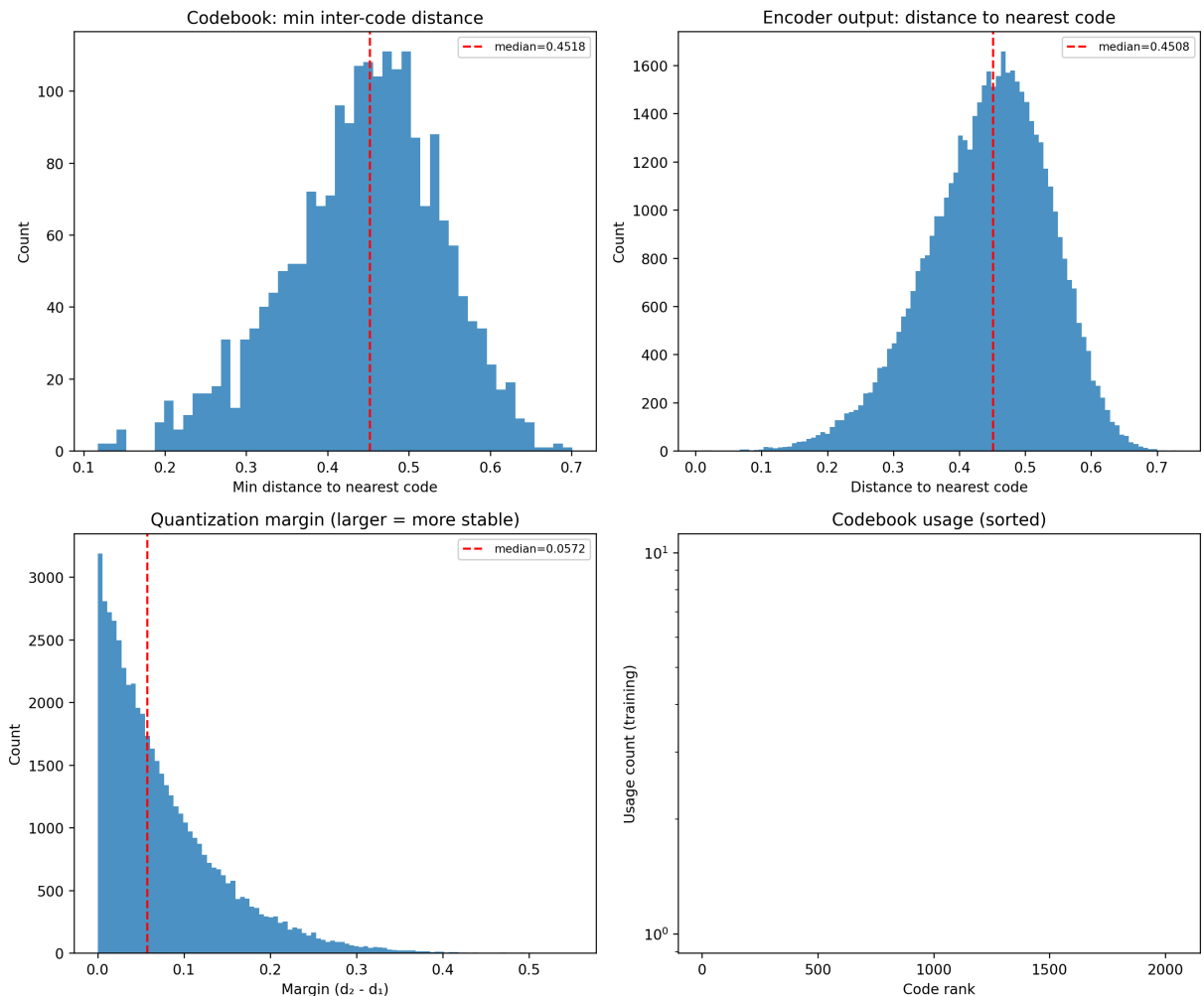
\_axes[1, 1].set\_yscale("log")

Inter-code dist - min: 0.1173, median: 0.4518, max: 0.7008

Margin (d2-d1) - mean: 0.0770, P5: 0.0043, P50: 0.0572

Dead codes (usage=0): 2048

Learnable scale (exp(log\_scale)): 1.0410



## 8. 入力記述子の分布

9D 記述子の各次元の分布を確認する。外れ値やマルチモーダル分布が量子化精度に影響している可能性。

特に CA 位置 (dim 0-2) はポケットサイズに依存して大きな値域を持つ。一方 N-CA / C-CA オフセット (dim 3-8) はバックボーンの局所構造なので 比較的コンパクトな分布になるはず。

```
In [ ]: dim_labels_full = [
    "CA_x",
    "CA_y",
    "CA_z",
    "N-CA_x",
    "N-CA_y",
    "N-CA_z",
    "C-CA_x",
    "C-CA_y",
    "C-CA_z",
]

_test_np = protein_test.cpu().numpy() # normalized
orig_np = orig # original scale

_fig, _axes = plt.subplots(3, 3, figsize=(14, 12))

for _i in range(9):
    ax = _axes[_i // 3, _i % 3]
    # Original scale distribution
    vals = orig_np[:, _i]
    ax.hist(vals, bins=100, edgecolor="none", alpha=0.7, density=True)
    ax.set_title(f"{dim_labels_full[_i]} (orig scale)")
    ax.set_xlabel("Value (Å)")
    ax.set_ylabel("Density")
    stats_text = (
        f"μ={vals.mean():.2f} σ={vals.std():.2f}\n"
        f"range=[{vals.min():.1f}, {vals.max():.1f}]"
    )
    ax.text(
        0.97,
        0.97,
        stats_text,
        transform=ax.transAxes,
        fontsize=7,
        ha="right",
        va="top",
        bbox={"boxstyle": "round", "facecolor": "wheat", "alpha": 0.5},
    )

_fig.suptitle(
    "Protein descriptor distribution (original scale)", fontsize=13, y=1.01
)
_fig.tight_layout()
```

```

# Print summary statistics
print("=== Descriptor statistics (original scale, Å) ===")
print(f"{'Dim':<10} {'Mean':>8} {'Std':>8} {'Min':>8} {'Max':>8} {'Range':>8}")
for _i, _label in enumerate(dim_labels_full):
    vals = orig_np[:, _i]
    print(
        f"{'_label':<10} {vals.mean():8.3f} {vals.std():8.3f} "
        f"{vals.min():8.3f} {vals.max():8.3f} {vals.ptp():8.3f}"
    )
print()
print("=== Descriptor statistics (normalized) ===")
print(f"{'Dim':<10} {'Mean':>8} {'Std':>8} {'Min':>8} {'Max':>8}")
for _i, _label in enumerate(dim_labels_full):
    vals = _test_np[:, _i]
    print(
        f"{'_label':<10} {vals.mean():8.4f} {vals.std():8.4f} "
        f"{vals.min():8.4f} {vals.max():8.4f}"
    )
_fig

```

```

=== Descriptor statistics (original scale, Å) ===

```

```

Dim          Mean      Std      Min      Max      Range

```

```

Traceback (most recent call last):

```

```

  File "", line 53, in <module>

```

```

    f"{vals.min():8.3f} {vals.max():8.3f} {vals.ptp():8.3f}"
                                ^^^^^^^

```

```

AttributeError: `ptp` was removed from the ndarray class in NumPy 2.0. Use n
p.ptp(arr, ...) instead.

```

## 9. 正規化空間での誤差分析

正規化後の空間でどの次元が再構成困難かを調べる。原点スケールでは CA 位置の誤差が大きく見えても、正規化後に均等なら単にスケールの問題。正規化後でも特定次元の誤差が大きければ、その次元の情報をエンコーダが十分に捉えていない。

```

In [ ]: dim_labels_norm = [
    "CA_x",
    "CA_y",
    "CA_z",
    "N-CA_x",
    "N-CA_y",
    "N-CA_z",
    "C-CA_x",
    "C-CA_y",
    "C-CA_z",
]

_test_np = protein_test.cpu().numpy()
_cont_np = cont_recon.cpu().numpy()
_vq_np = vq_recon.cpu().numpy()

cont_per_dim_mse = np.mean((_test_np - _cont_np) ** 2, axis=0)
vq_per_dim_mse = np.mean((_test_np - _vq_np) ** 2, axis=0)

```



```

quant_per_dim_mse = (
    vq_per_dim_mse - cont_per_dim_mse
) # quantization-only contribution

_fig, _axes = plt.subplots(1, 2, figsize=(14, 5))

x = np.arange(9)
w = 0.35
_axes[0].bar(
    x - w / 2,
    cont_per_dim_mse,
    w,
    label="Continuous (encoder capacity)",
    color="#3498db",
)
_axes[0].bar(
    x + w / 2,
    vq_per_dim_mse,
    w,
    label="VQ (capacity + quantization)",
    color="#e74c3c",
)
_axes[0].set_xticks(x, dim_labels_norm, rotation=45, ha="right", fontsize=8)
_axes[0].set_ylabel("MSE (normalized)")
_axes[0].set_title("Per-dim MSE: Continuous vs VQ (normalized space)")
_axes[0].legend(fontsize=8)

# Stacked: encoder error + quantization error
_axes[1].bar(x, cont_per_dim_mse, label="Encoder capacity error", color="#3498db")
_axes[1].bar(
    x,
    np.maximum(quant_per_dim_mse, 0),
    bottom=cont_per_dim_mse,
    label="Quantization error",
    color="#e74c3c",
    alpha=0.7,
)
_axes[1].set_xticks(x, dim_labels_norm, rotation=45, ha="right", fontsize=8)
_axes[1].set_ylabel("MSE (normalized)")
_axes[1].set_title("Error decomposition: encoder capacity + quantization")
_axes[1].legend(fontsize=8)

_fig.tight_layout()

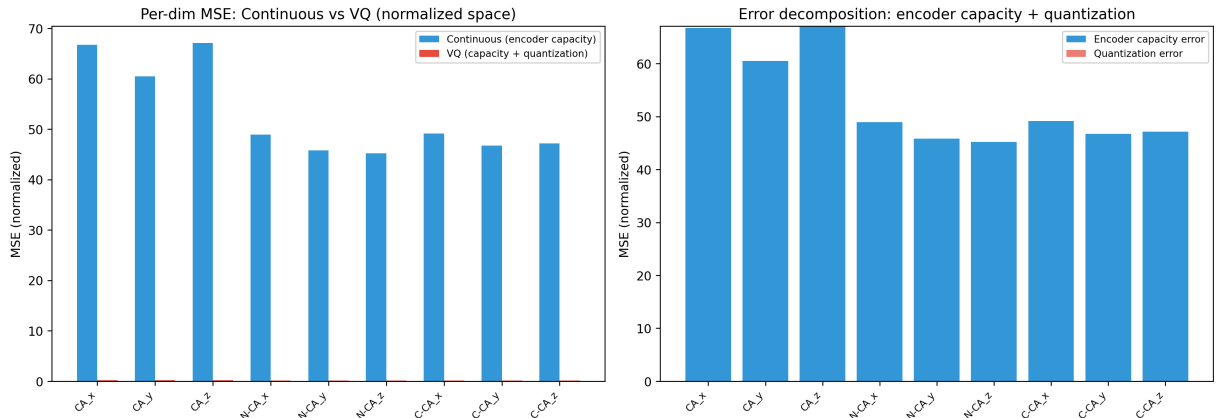
print("=== Per-dim MSE (normalized space) ===")
print(f"{'Dim':<10} {'Continuous':>12} {'VQ':>12} {'Quant-only':>12}")
for _i, _label in enumerate(dim_labels_norm):
    print(
        f"{'_label':<10} {cont_per_dim_mse[_i]:12.6f} {vq_per_dim_mse[_i]:12.6f} {quant_per_dim_mse[_i]:12.6f}"
    )
_fig

```

```

=== Per-dim MSE (normalized space) ===
Dim          Continuous      VQ      Quant-only
CA_x         66.709320      0.202114 -66.507210
CA_y         60.497242      0.208946 -60.288296
CA_z         67.063820      0.219330 -66.844490
N-CA_x       48.930084      0.142714 -48.787369
N-CA_y       45.805557      0.148139 -45.657417
N-CA_z       45.174431      0.145080 -45.029350
C-CA_x       49.121151      0.142481 -48.978672
C-CA_y       46.741116      0.149560 -46.591557
C-CA_z       47.175156      0.147483 -47.027672

```



## 10. 潜在空間の次元利用率

16D 潜在空間の各次元がどの程度使われているかを調べる。一部の次元しか使われていない場合、実質的な潜在次元が小さく、表現力が足りていない可能性がある。

```

In [ ]: z_raw = z_all.cpu().numpy()
        z_l2 = F.normalize(z_all, p=2, dim=-1).cpu().numpy()

        _fig, _axes = plt.subplots(1, 3, figsize=(16, 4))

        # Variance per latent dimension (raw)
        z_var = np.var(z_raw, axis=0)
        z_var_sorted_idx = np.argsort(z_var)[::-1]
        _axes[0].bar(range(len(z_var)), z_var[z_var_sorted_idx])
        _axes[0].set_xlabel("Latent dimension (sorted by variance)")
        _axes[0].set_ylabel("Variance")
        _axes[0].set_title("Encoder output: variance per latent dim (raw)")
        for _i, _idx in enumerate(z_var_sorted_idx):
            _axes[0].text(_i, z_var[_idx], f"d{_idx}", ha="center", va="bottom", for

        # Variance per latent dimension (L2-normalized)
        z_l2_var = np.var(z_l2, axis=0)
        z_l2_var_sorted_idx = np.argsort(z_l2_var)[::-1]
        _axes[1].bar(range(len(z_l2_var)), z_l2_var[z_l2_var_sorted_idx])
        _axes[1].set_xlabel("Latent dimension (sorted by variance)")
        _axes[1].set_ylabel("Variance")
        _axes[1].set_title("Encoder output: variance per latent dim (L2-norm)")
        for _i, _idx in enumerate(z_l2_var_sorted_idx):
            _axes[1].text(

```

```

        _i, z_l2_var[_idx], f"d{_idx}", ha="center", va="bottom", fontsize=6
    )

    # Cumulative explained variance ratio (PCA-style)
    total_var = z_l2_var.sum()
    cum_var = np.cumsum(z_l2_var[z_l2_var_sorted_idx]) / total_var
    _axes[2].plot(range(1, len(cum_var) + 1), cum_var, "o-", markersize=4)
    _axes[2].set_xlabel("Number of dimensions")
    _axes[2].set_ylabel("Cumulative variance ratio")
    _axes[2].set_title("Latent dimension utilization (L2-norm)")
    _axes[2].axhline(0.9, color="r", linestyle="--", alpha=0.5, label="90%")
    _axes[2].axhline(0.95, color="orange", linestyle="--", alpha=0.5, label="95%")
    _axes[2].legend(fontsize=8)
    _axes[2].grid(visible=True, alpha=0.3)

    _fig.tight_layout()

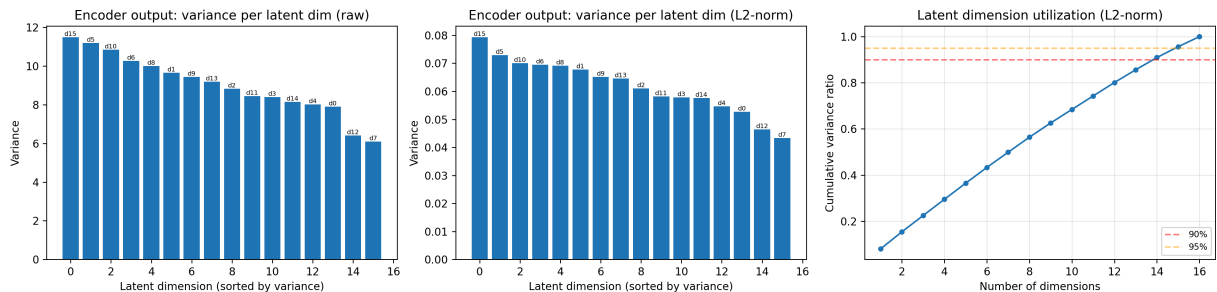
    n_for_90 = np.searchsorted(cum_var, 0.9) + 1
    n_for_95 = np.searchsorted(cum_var, 0.95) + 1
    print(f"Dims for 90% variance: {n_for_90} / {len(z_l2_var)}")
    print(f"Dims for 95% variance: {n_for_95} / {len(z_l2_var)}")
    print(
        f"Effective dimensionality is limited – {n_for_90} dims capture 90% of t
    )
    _fig

```

Dims for 90% variance: 14 / 16

Dims for 95% variance: 15 / 16

Effective dimensionality is limited – 14 dims capture 90% of the variance



## 11. 診断サマリー

上の結果を踏まえた考察用メモ。

### チェックリスト

| 仮説        | 確認セル                  | 結論                                 |
|-----------|-----------------------|------------------------------------|
| エンコーダ容量不足 | セル 2 (cont vs VQ MSE) | cont MSE $\approx$ VQ MSE なら容量不足   |
| 量子化誤差     | セル 2, 4 (margin)      | VQ MSE $\gg$ cont MSE なら量子化がボトルネック |
| CA位置のスケール | セル 3 (per-dim RMSE)   | CA dim が支配的か                       |

| 仮説                | 確認セル                       | 結論                         |
|-------------------|----------------------------|----------------------------|
| 残基間コンテキスト         | セル 6 (RMSE vs pocket size) | 大きいポケットで誤差大 → コンテキスト必要     |
| Codebook collapse | セル 7 (usage, margin)       | 少数コードに集中 → collapse        |
| 潜在次元不足            | セル 10 (variance)           | 少数dimに分散集中 → latent_dim 不足 |

## 次のアクション候補

- **A:** Transformer 導入 → セル 6 で pocket size 相関が高ければ有効
- **B:** 記述子拡張 → セル 3 でオフセット誤差が大きければ側鎖情報追加
- **C:** RVQ/PQ → セル 2 で量子化がボトルネック、セル 7 で margin 小なら有効
- **D:** encoder 拡大 → セル 2 で容量がボトルネック、セル 10 で有効次元少なければ